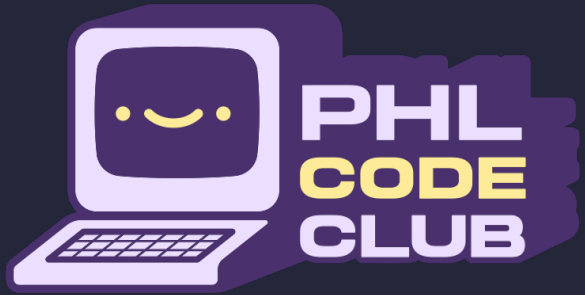
Cache Me If You Can Outrunning Slow Web Requests

echo $(whoami)

- Principle Software Engineer*
- Mechanical keyboard and ergonomics enthusiast
- Die hard terminal fanboy
- Functional programming nerd
- Co-founder of the PHL Code Club
  - I have stickers =3



Here is me with me cat, Fritzy =3

*Technically I'm still a Senior

On the docket


Computer memory 101


Average web app structure


Why cache in web services?


Following a web request through the service

- Browser/client caching
- CDNs and edge caching
- Application caching


Caching strategies and pitfalls

Cache terminology 101

Common caching terms you'll hear

Cache outcomes

- **Hit**: Data found in cache - fast response
- **Miss**: Data not in cache - need to fetch from origin
- **Stale**: Cached data is outdated but still served
- **Fresh**: Cached data is current and valid

Cache management

- **TTL (Time To Live)**: How long data stays cached
- **Invalidation**: Marking cached data as outdated
- **Eviction/Purge/Clear**: Removing data from cache
- **CDN (Content Delivery Network)**: Global cache network
- **Edge**: Compute platforms that live in CDN Points of Presence (POP)

Computer memory 101

Types of computer memory

Persistent

- Long term storage
- Slow (comparatively)
- Does not require power to retain data

Volatile

- Short term storage
- Fast(er)
  - Depends on the specifics of the volatile memory
  - RAM is faster than persistent drives thanks to bandwidth and physical location near the CPU
  - CPU caches (HE SAID THE WORD) are faster than RAM due to colocation with the CPU, but they are *tiny* in comparison
- Requires power to retain information, once power is lost, all data is lost

We'll mostly focus on persistent storage on disk/SSD and RAM for volatile memory.

Modern web architectures

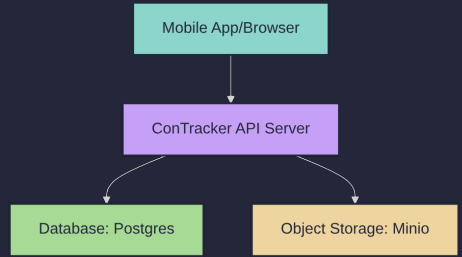
Main components for ConTracker

Client

- Web app, mobile app, or even other servers.
- Requests data from the API server, such as user profiles, messages, etc.

API Server

- Small HTTP server.
- Handles requests for data from clients.



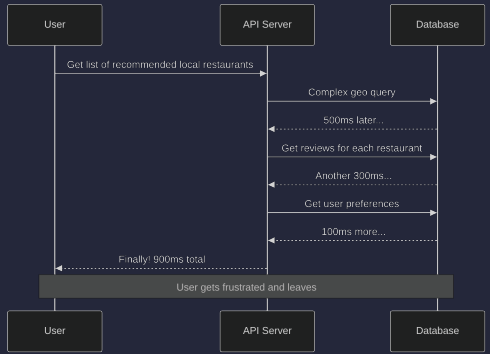
Storage

- Database
  - Stores structured data such as user profiles, relationships between users, etc.
- Object Storage
  - Stores large objects such as photos, audio files, and videos.

But why spend my cash on cache?

The problem with no caching

Requests to the API server *without* caching might look something like this:

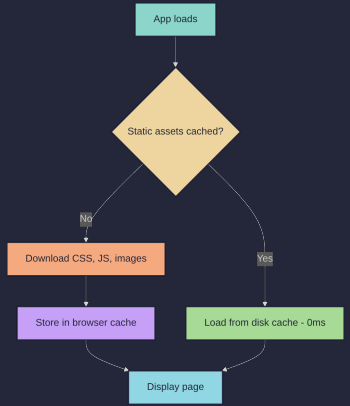


The numbers that hurt

- Network latency: 50-200ms per request
- Database queries: 100-500ms for complex operations
- Users expect responses under 100ms
- Every 100ms delay = 1% drop in sales [1]

Browser caching (automatic)

Your browser is already caching lots of data automatically*:



What gets cached automatically

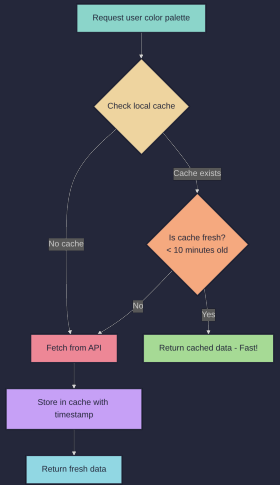
- JavaScript bundles (maybe a SPA)
- CSS stylesheets
- Photos
- Icons
- Fonts

HTTP response headers control this behavior

```
# Cache for 1 day
Cache-Control: public, max-age=86400
# Version identifier
ETag: "company-logo-v2.png"
```

Client caching (manual)

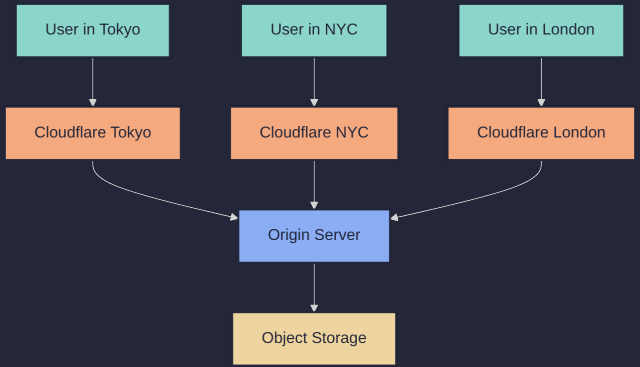
We can also cache API responses in our client code. This is perfect for high priority data that might not change often, or is readily controlled by the user.



CDNs - Your global cache network

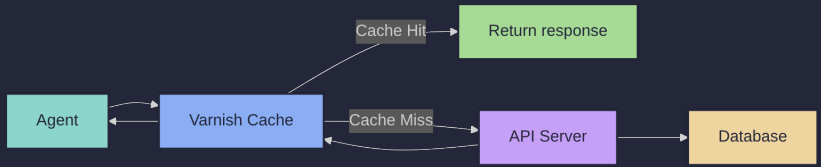
What do CDN's generally cache

- Web app assets
  - HTML
  - CSS
  - JS
- Static content
  - Logos
  - Fonts
  - Public Documents



HTTP Cache (Varnish, Nginx w/ plugins, etc)

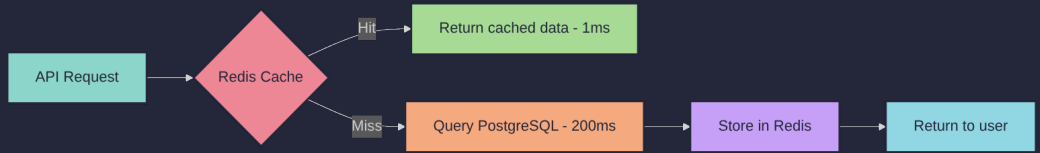
HTTP reverse proxy that sits in front of ConTracker:



Perfect for caching entire HTTP responses on the edge!

Application caches - Redis, Memcached, Valkey, et al

Where API servers store frequently accessed data:



How it works

- **Cache key strategy**: Use consistent naming like `user_prefs:{user_id}` and `post:{post_id}`
- **TTL management**: Different timeframes for different data types
- **Cache warming**: Pre-populate frequently accessed data
- **Fallback handling**: Always handle cache misses gracefully

Cache invalidation - The hard problem

> "There are only two hard things in Computer Science: cache invalidation, naming things, and off-by-one errors."

Cache invalidation strategies

Time-based (TTL) - Simple

Set expiration times when storing data. Post info gets 1 hour TTL, user sessions get 30 minutes. Easy to implement but might serve stale data.

Event-based - More complex but accurate

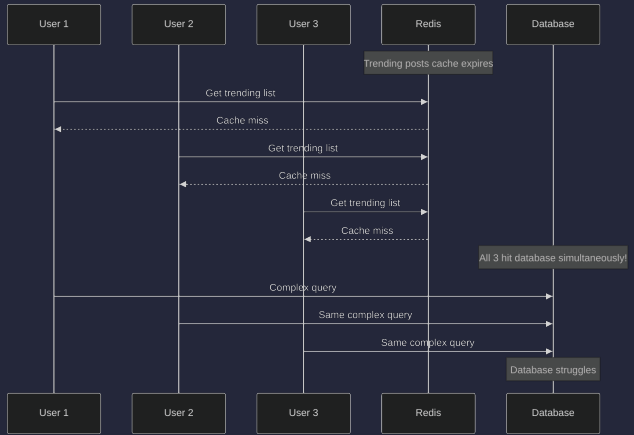
When data changes, immediately invalidate related caches. Post edits trigger cache clears for that post and all search results. Keeps data fresh but requires more coordination.

Manual - For emergencies

Admin endpoints to clear caches when things go wrong. The "nuclear option" when you need to clear things quickly.

The cache stampede problem

What happens when the API's "trending posts" cache expires:



The cache stampede solution

Lock-based approach

First request gets a lock, computes the data, and stores it. Other requests wait for the first one to finish and return the data that is stored in the cache.

Also called cache collapse or request coalescing [2]

Probabilistic refresh

Refresh cache before it expires based on random probability. Spreads out the load instead of everyone hitting at once.

Common pitfalls (and how we avoided them)

Cache consistency nightmare

- **Problem:** CDN shows old case status, app cache shows new status
- **Solution:** Version-based invalidation and cache warming

Over-caching everything

- **Problem:** Caching constantly updating information can lead to consistently stale data
- **Solution:** Profile first, then cache selectively

Under-caching the obvious

- **Problem:** Didn't cache document OCR operations - server melting
- **Solution:** Cache expensive operations, not just database queries

Default TTL disaster

- **Problem:** Used default TTL for everything
- **Solution:** Different TTLs for different data types

Best practices

Do ✅

- **Set appropriate TTLs:** Posts = 1 hour, sessions = 30 minutes
- **Monitor cache hit ratios:** Aim for >80% on critical paths
- **Use consistent cache keys:** `user_prefs:{user_id}`, not `userprefs{id}`
- **Handle cache failures gracefully:** App should work even if cache is down
- **Warm critical caches:** Pre-populate high-profile users posts

Don't ❌

- **Cache user-specific data globally:** Don't put user preferences in CDN
- **Ignore cache stampede:** Popular endpoints need protection
- **Cache sensitive information:** PII, sensitive documents, etc.
- **Cache everything:** Caching rarely-accessed data wastes memory

Summary

Start simple

- Browser cache + CDN covers 80% of performance gains
- Add app cache when database becomes the bottleneck
- Add edge cache when distributing API responses globally
- Don't over-engineer early

Measure everything

- You can't optimize what you don't measure
- Cache hit ratios matter more than cache size
- Monitor P95/P99 response times, not just averages

Layer strategically

- Each cache serves different needs and timeframes
- Static assets → CDN (long TTL)
- API responses → Edge cache (medium TTL)
- Expensive computations → App cache (short TTL)

Invalidate based on needs

- TTL-based is simple but can serve stale data
- Event-based is complex but keeps data fresh
- Have a manual override for emergencies

Questions?

Citations

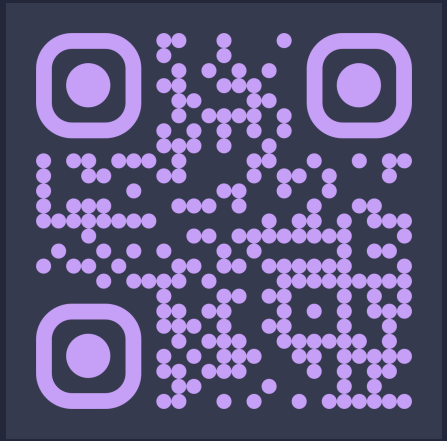
1. **Web performance affects sales (** *https://glinden.blogspot.com/2006/12/slides-from-my-talk-at-stanford.html*)
2. **Cache Collapse/Request Coalescing (** *https://www.fastly.com/documentation/guides/concepts/edge-state/cache/request-collapsing/*)

Resources for your caching journey

- **Redis Documentation** (*https://redis.io/docs/*) - Comprehensive guide
- **Cloudflare Cache Docs** (*https://developers.cloudflare.com/cache/*) - CDN best practices
- **Varnish Book** (*https://book.varnish-software.com/*) - Advanced HTTP caching
- **High Performance Browser Networking** (*https://hpbn.co/*) - Free online book

Connect with me :)

Me:



PHL Code Club: